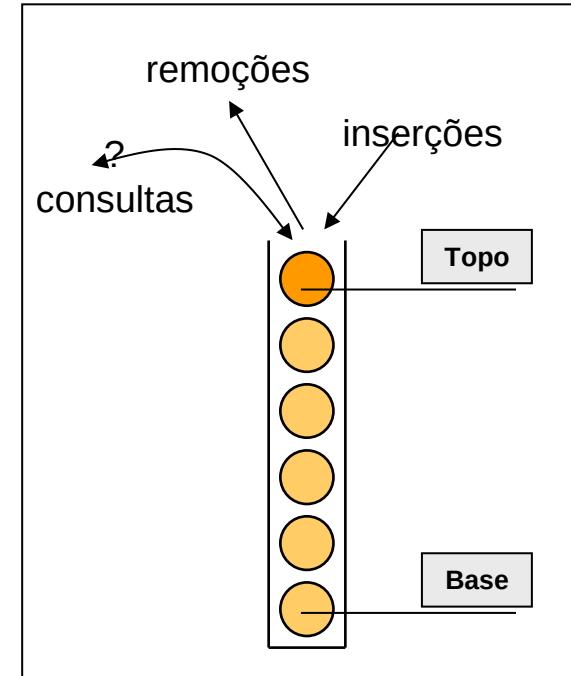
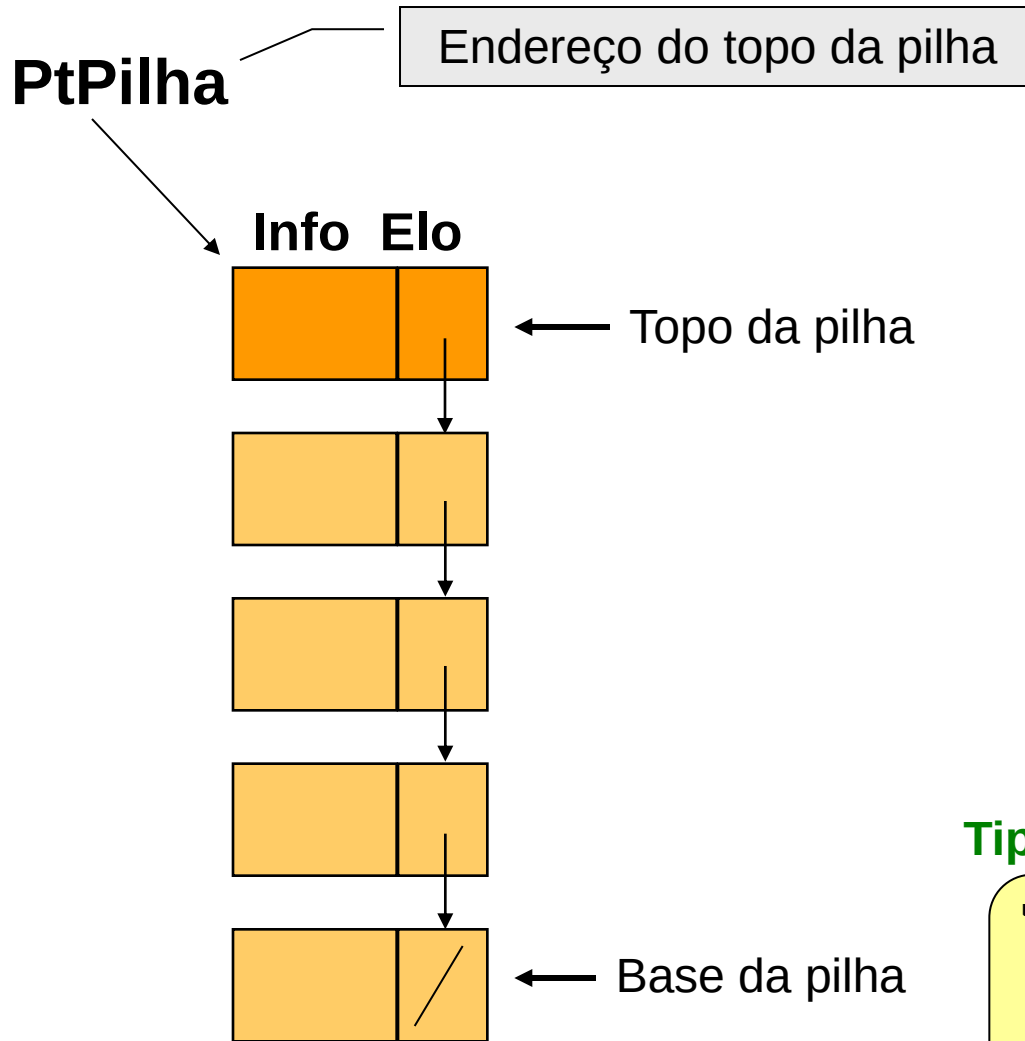


Pilhas

Pilhas implementadas por encadeamento

Pilha implementada por encadeamento



Tipo de dados utilizado nos algoritmos:

```
TipoNode = registro
    Info: TipoInfo
    Elo: TipoPtNode
fim registro
```

Criação de pilha encadeada

- Pilha criada vazia
- Atribuir endereço nulo para apontador que contém o endereço do topo da pilha

Algoritmo:

Criar Pilha Encadeada

Algoritmo **CriarPilhaEnc**

Entradas: -

Saída: PtPilha (TipoPtNodo)

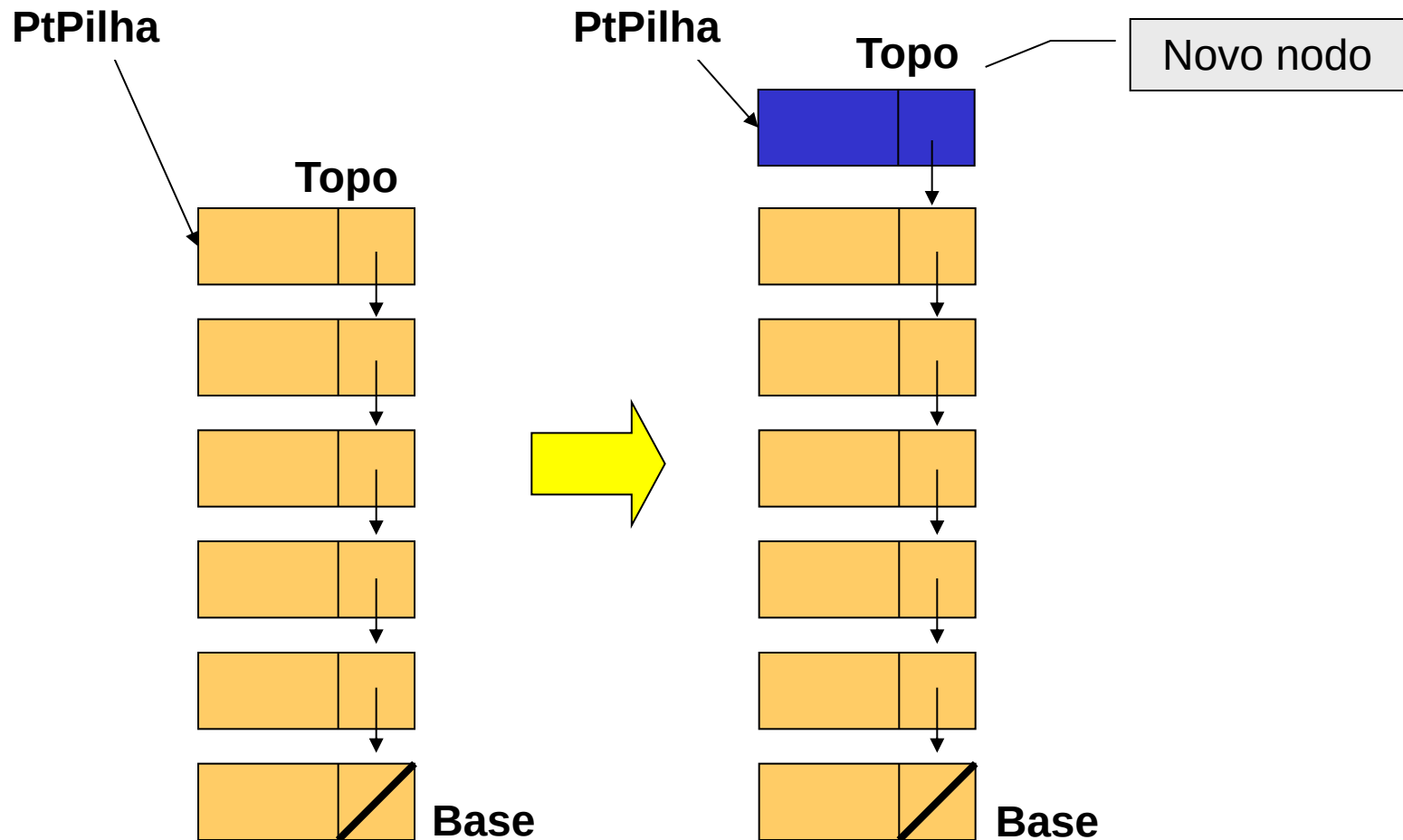
início

PtPilha $\leftarrow$ nulo

fim

Inserção de um nodo em pilha encadeada

- Novo nodo inserido sempre no topo da pilha



Algoritmo:

Inserir nodo em Pilha Encadeada

Algoritmo **InserirPilhaEnc**

Entradas: PtPilha (TipoPtNodo)

Valor (TipoInfo)

Saída: PtPilha (TipoPtNodo)

Variável local: PtNovo (TipoPtNodo)

início

alocar(PtNovo)

PtNovo[↑].Info ← Valor

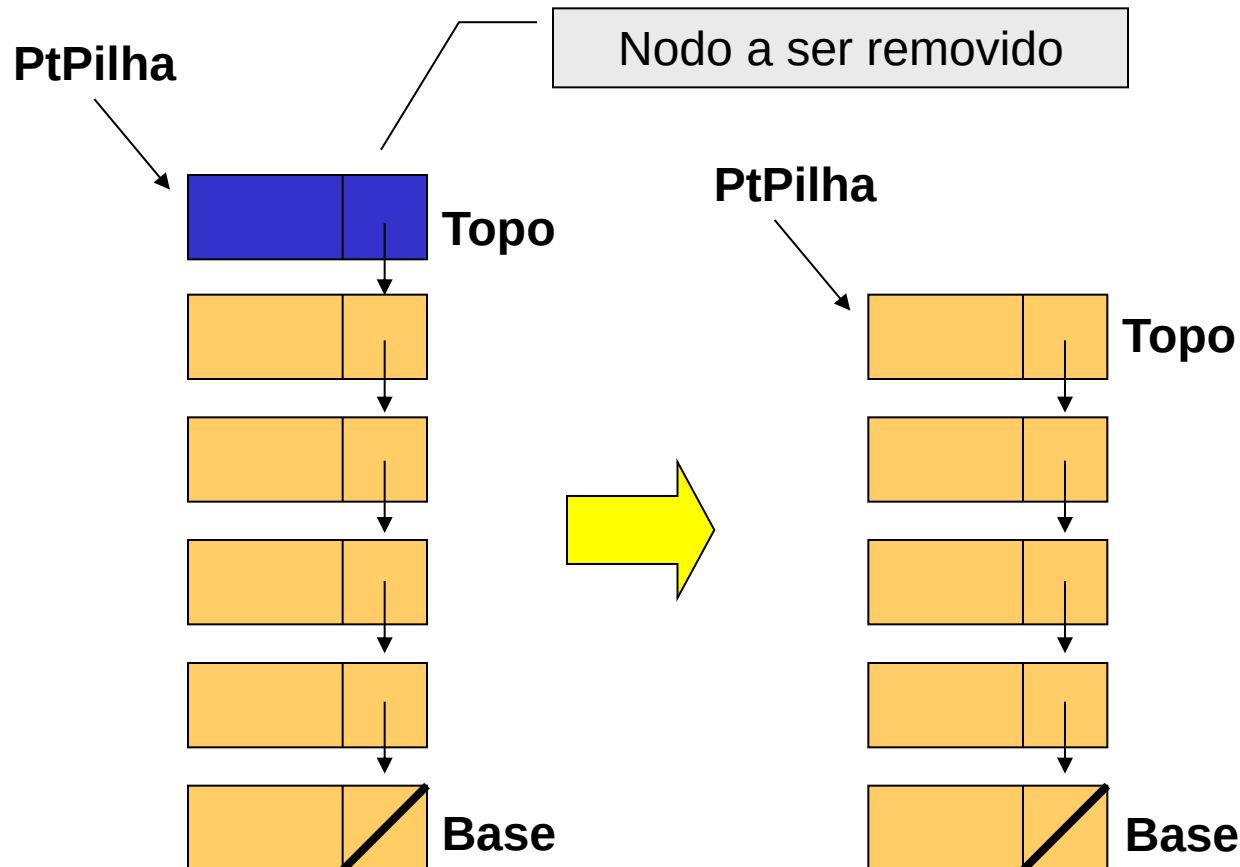
PtNovo[↑].Elo ← PtPilha

PtPilha ← PtNovo

fim

Remoção de um nodo de uma pilha encadeada

- Só pode ser removido o nodo do topo da pilha



Algoritmo:

Remover nodo de Pilha Encadeada

Algoritmo **RemoverPilhaEnc**

Entrada: PtPilha (TipoPtNodo)

Saídas: PtPilha (TipoPtNodo)

Sucesso (lógico)

Variável local: PtAux (TipoPtNodo)

início

se PtPilha $\neq$ nulo

então início

PtAux $\leftarrow$ PtPilha

PtPilha $\leftarrow$ PtPilha $\uparrow$.Elo

liberar(PtAux)

Sucesso $\leftarrow$ verdadeiro

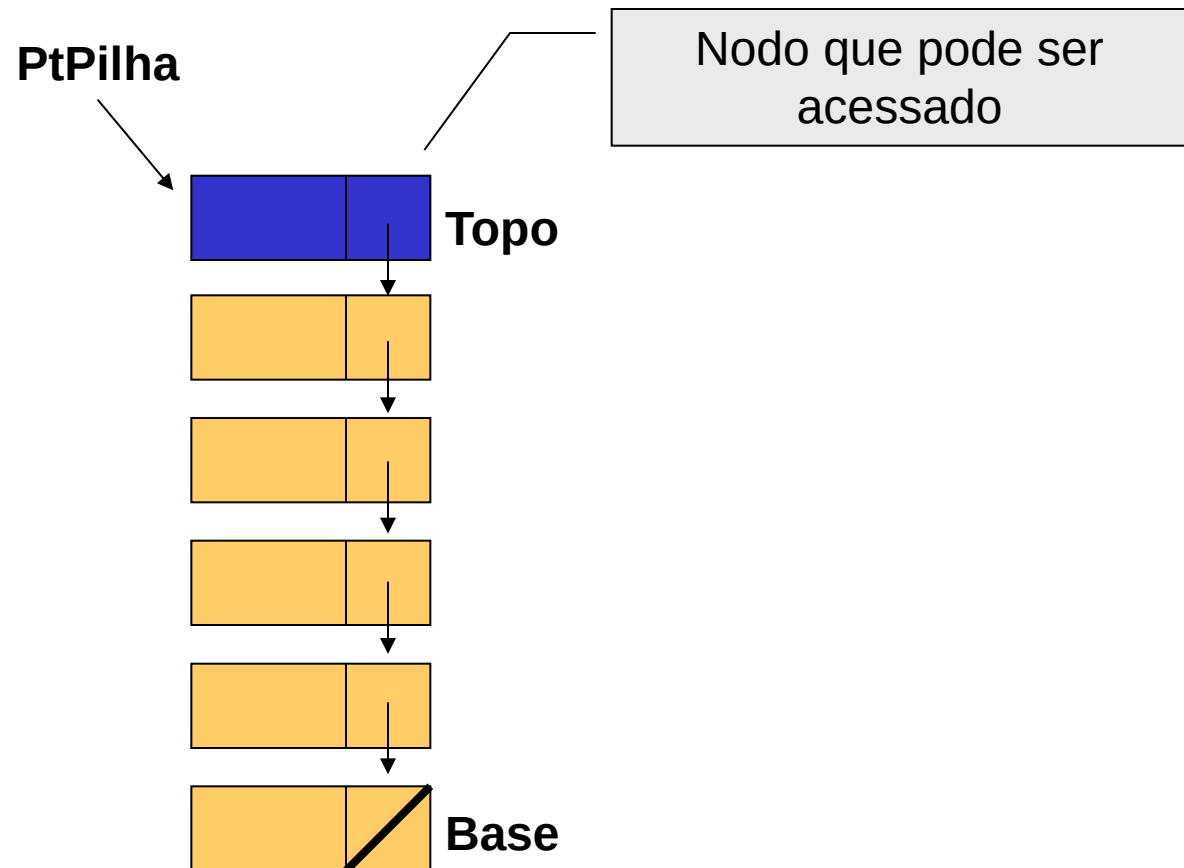
fim

senão Sucesso $\leftarrow$ falso

fim

Acesso à pilha encadeada

- Só pode ser acessado o nodo do topo da pilha



Algoritmo:

Consultar nodo do topo de Pilha Encadeada

```
Algoritmo ConsultarPilhaEnc
  Entrada: PtPilha (TipoPtNodo)
  Saídas: Valor (TipoInfo)
          Sucesso (lógico)

início
  se PtPilha = nulo
  então Sucesso ← falso
  senão início
    Sucesso ← verdadeiro
    Valor ← PtPilha↑.Info
  fim
fim
```

Algoritmo: Desempilhar

Consulta nodo do topo da pilha, e o remove da pilha

Algoritmo Desempilhar

Entrada: PtPilha (TipoPtNodo)

Saídas: PtPilha (TipoPtNodo)

Valor (TipoInfo)

Sucesso (lógico)

Variável local: PtAux (TipoPtNodo)

início

se PtPilha = nulo

então Sucesso ← falso

senão início

Sucesso ← verdadeiro

Valor ← PtPilha↑.Info

PtAux ← PtPilha

PtPilha ← PtPilha↑.Elo

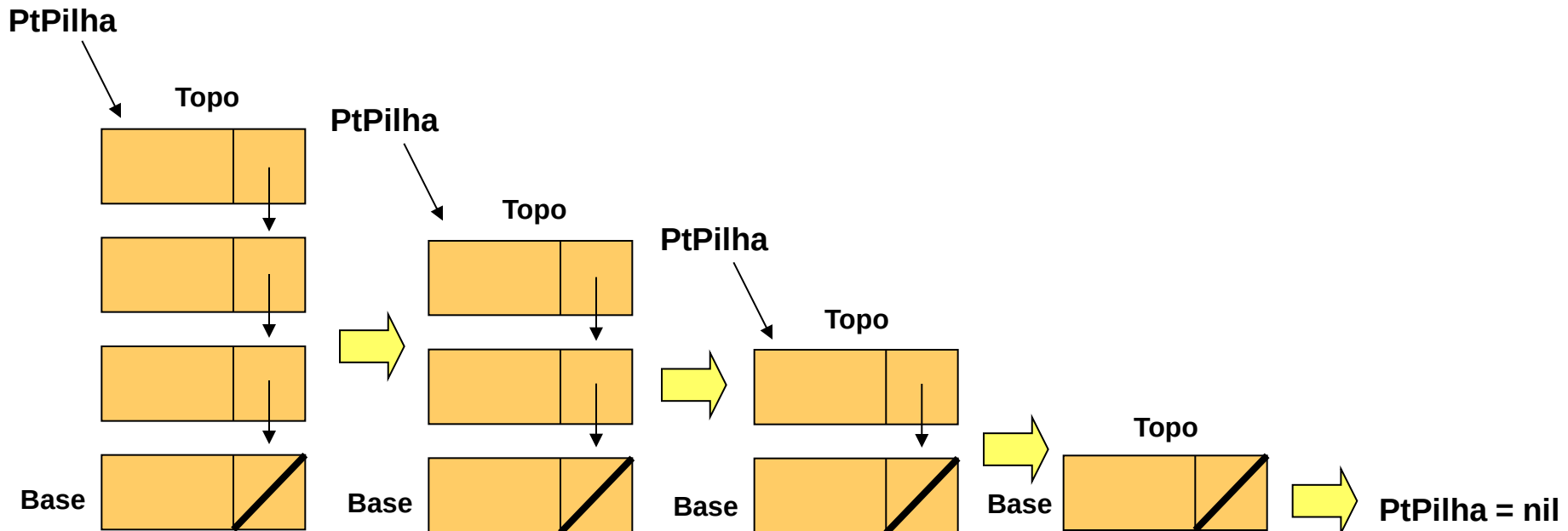
liberar(PtAux)

fim

fim

Destruição de uma pilha encadeada

- Liberar espaço ocupado pelos nodos, sempre a partir do topo da pilha
- No final: apontador para o topo = endereço nulo



Algoritmo:

Destruir Pilha Encadeada

Algoritmo DestruirPilhaEnc

Entrada: PtPilha (TipoPtNodo)

Saída: PtPilha (TipoPtNodo)

Variável local: PtAux (TipoPtNodo)

início

enquanto PtPilha $\neq$ nulo

faça início

PtAux $\leftarrow$ PtPilha

PtPilha $\leftarrow$ PtPilha $\uparrow$.Elo

liberar(PtAux)

fim

fim

Filas

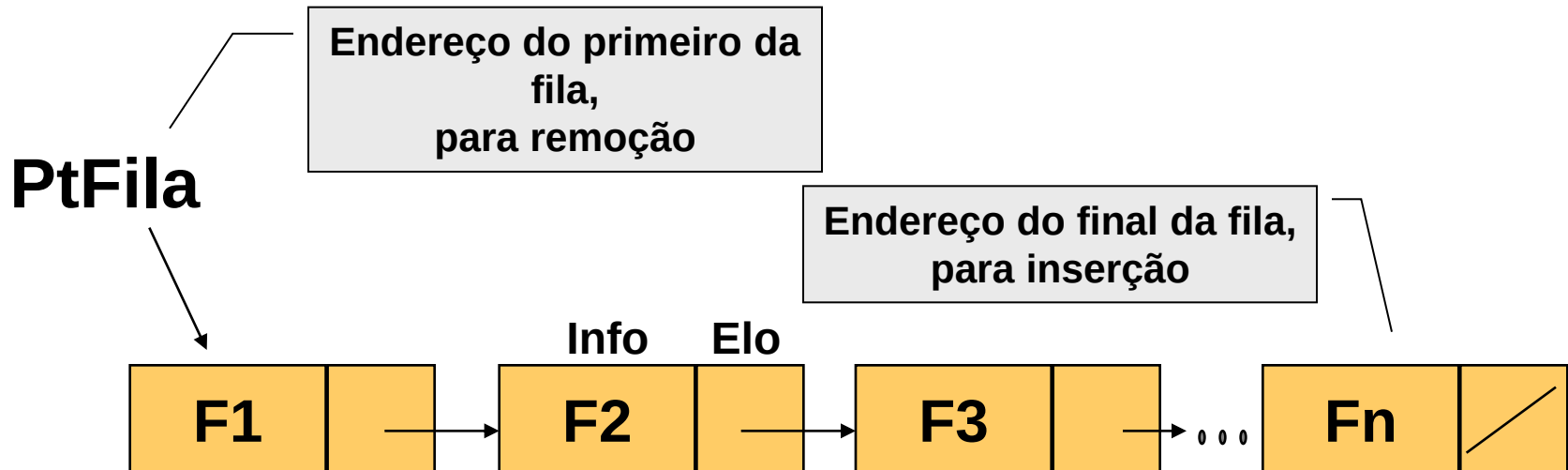
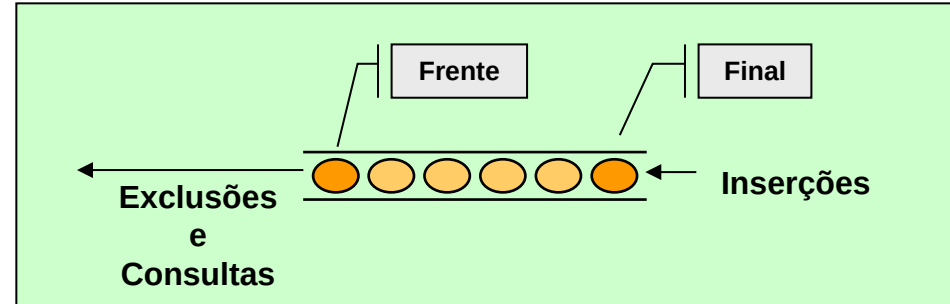
Filas implementadas por encadeamento

Filas implementadas por encadeamento

Tipo de dados para nodos da fila:

```

TipoNode = registro
    Info: TipoInfo
    Elo: TipoPtNode
fim registro
  
```



Para acessar o último nodo, é necessário percorrer toda a fila a partir do primeiro nodo

Filas por encadeamento com descritor

Descritor

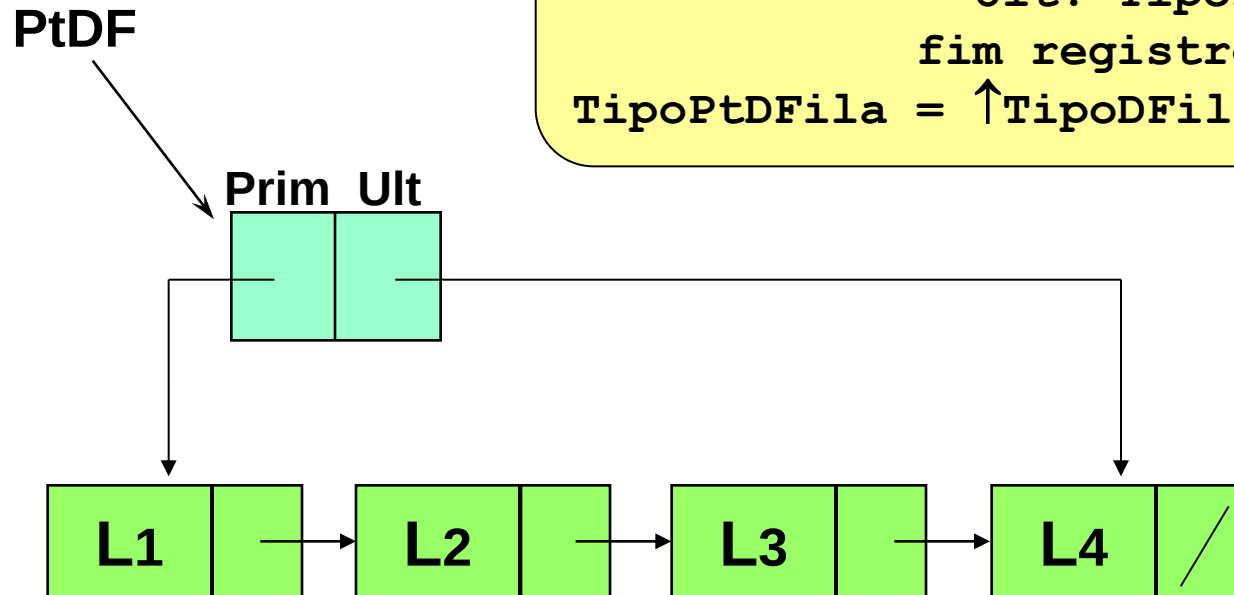
Prim: primeiro da fila

Ult : último da fila

Tipo de dados para o descritor da fila:

```

TipoDFila = registro
    Prim: TipoPtNode
    Ult: TipoPtNode
fim registro
TipoPtDFila = ↑TipoDFila
  
```



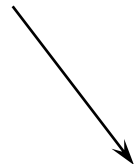
Operações sobre Filas implementadas por encadeamento com descritor

- Criar uma fila vazia
- Inserir um nodo no final da fila
- Excluir o nodo do início da fila
- Consultar / modificar nodo do início da fila
- Destruir a fila

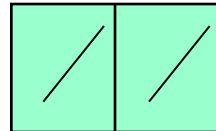
Criação da fila encadeada

- Alocar o descritor da fila
- Descritor inicializado em endereços nulos
- Fila vazia

PtDFila



Prim Ult



Algoritmo:

Criar Fila Encadeada endereçada por descritor

Algoritmo CriarFilaEnc

Entradas: -

Saída: PtDFila (TipoPtDFila)

início

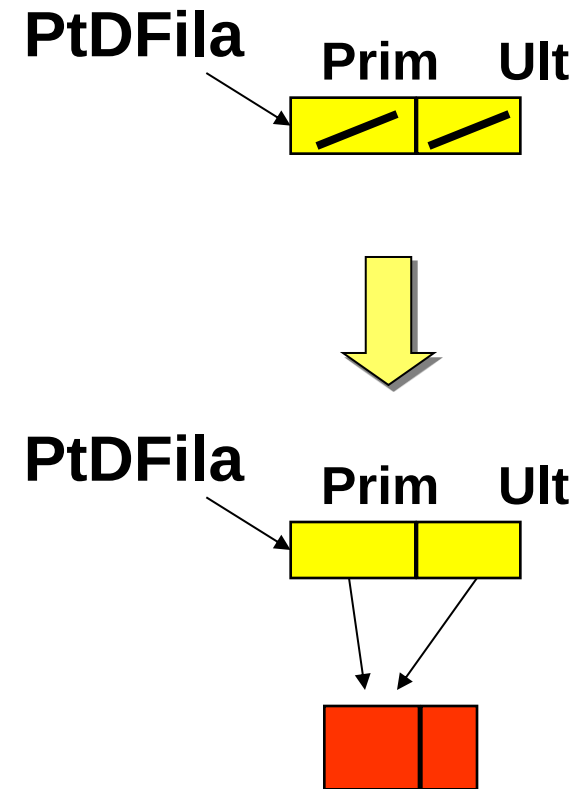
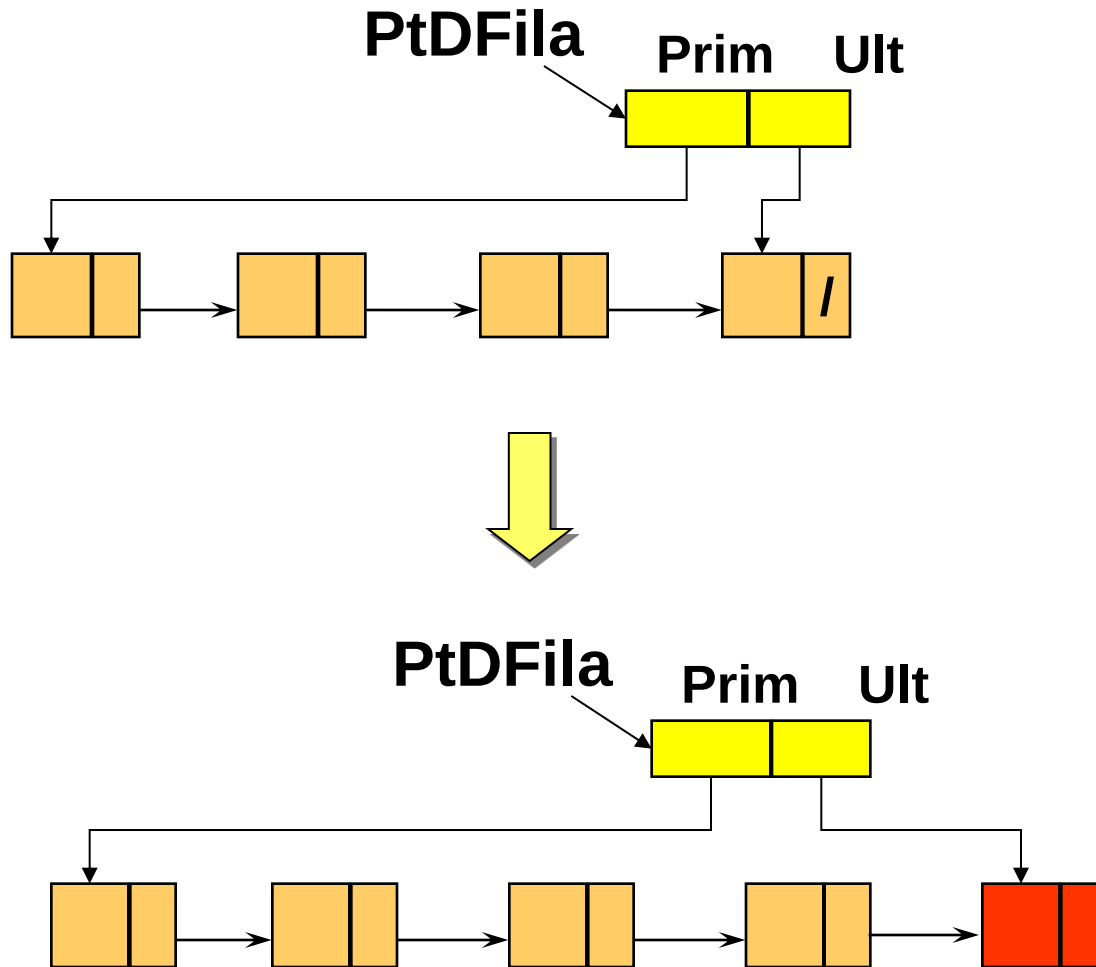
alocar(PtDFila)

PtDFila[↑].Prim ← nulo

PtDFila[↑].Ult ← nulo

fim

Inserção de um nodo na fila encadeada



Algoritmo:

Inserir novo nodo em Fila Encadeada endereçada por descritor

Algoritmo **InserirFilaEnc**

Entradas: PtDFila (TipoPtDFila)

Valor (TipoInfo)

Saídas: -

Variável auxiliar: PtNovo (TipoPtNodo)

início

alocar(PtNovo)

PtNovo[↑].Info ← Valor

PtNovo[↑].Elo ← nulo

se PtDFila[↑].Prim = nulo

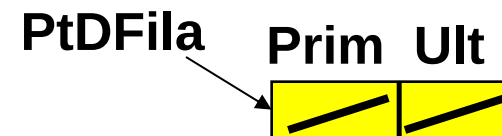
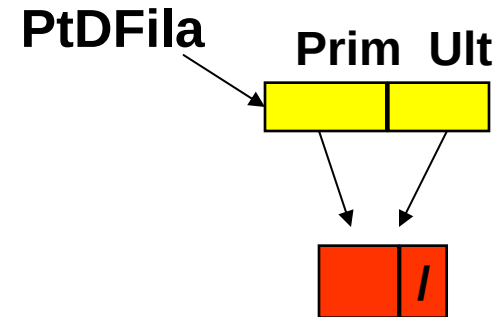
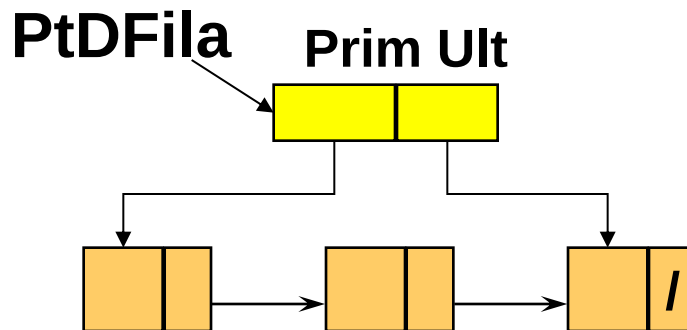
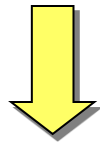
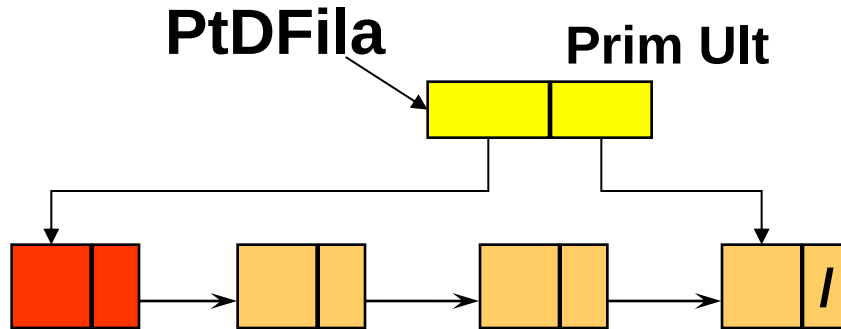
então PtDFila[↑].Prim ← PtNovo

senão (PtDFila[↑].Ult)[↑].Prox ← PtNovo

PtDFila[↑].Ult ← PtNovo

fim

Remoção de um nodo de fila encadeada



Algoritmo:

Remover um nodo de Fila Encadeada endereçada por descritor

Algoritmo **RemoverFilaEnc**

Entrada: PtDFila (TipoPtDFila)

Saída: Sucesso (lógico)

Variável auxiliar: PtAux (TipoPtNodo)

início

se PtDFila↑.Prim ≠ nulo

então início

PtAux ← PtDFila↑.Prim

PtDFila↑.Prim ← PtAux↑.Elo

liberar(PtAux)

se PtDFila↑.Prim = nulo

então PtDFila↑.Ult ← nulo

Sucesso ← verdadeiro

fim

senão Sucesso ← falso

fim

Algoritmo:

Consultar Fila Encadeada endereçada por descritor

Algoritmo ConsultarFilaEnc

Entrada: PtDFila (TipoPtDFila)

Saídas: Valor (TipoInfo)

Sucesso (lógico)

início

se PtDFila[↑].Prim ≠ nulo

então início

Valor ← PtDFila[↑].Prim[↑].Info

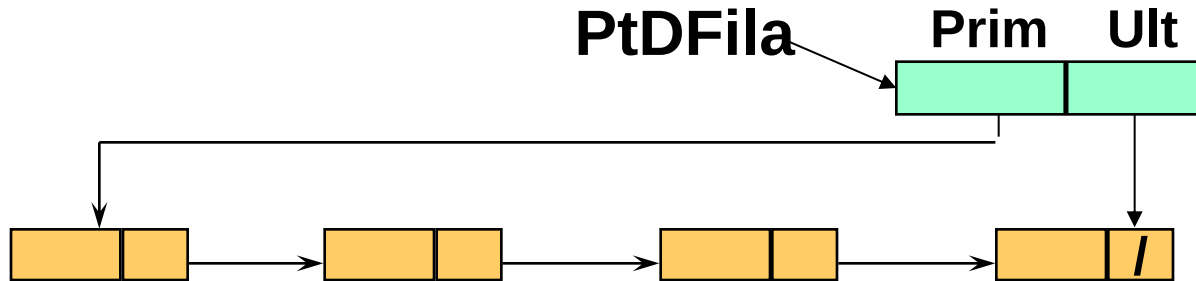
Sucesso ← verdadeiro

fim

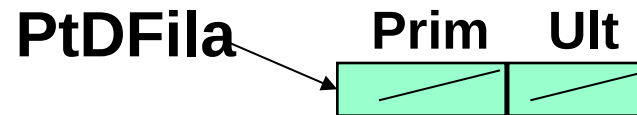
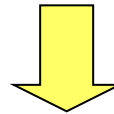
senão Sucesso ← falso

fim

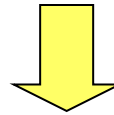
Destruição de fila encadeada



**Liberar posições
ocupadas pela
lista**



Liberar descritor



PtDFila = nulo

Algoritmo:

Destruir Fila Encadeada endereçada por descritor

Algoritmo DestruirFilaEnc

Entrada: PtDFila (TipoPtDFila)

Saída: PtDFila (TipoPtDFila)

Variáveis auxiliares: P1, P2 (TipoPtNodo)

início

se PtDFila↑.Prim ≠ nulo

então início

 P1 ← PtDFila↑.Prim

 repita

 P2 ← P1↑.Elo

 liberar(P1)

 P1 ← P2

 até P1 = nulo

 fim

liberar(PtDFila)

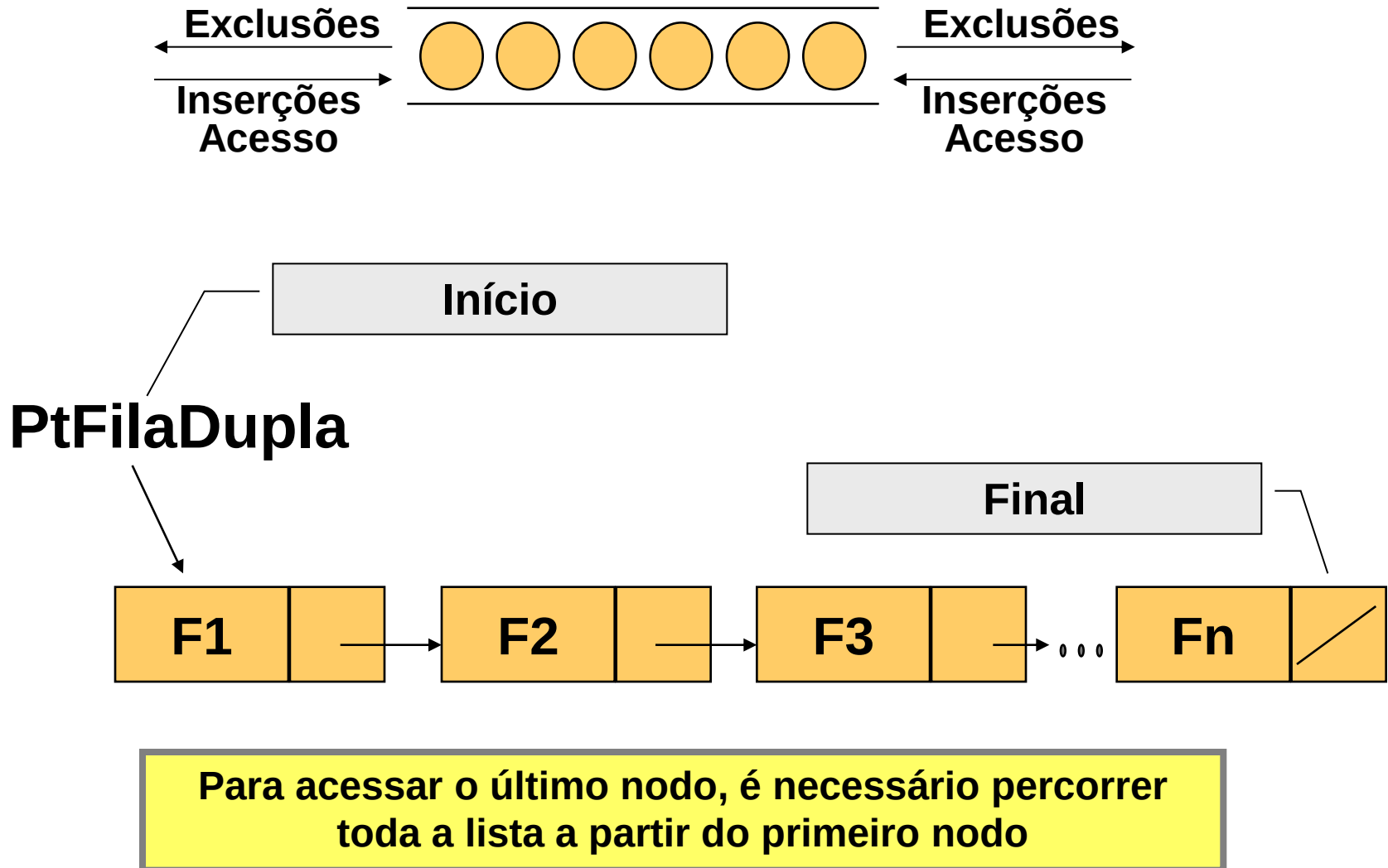
PtDFila ← nulo

fim

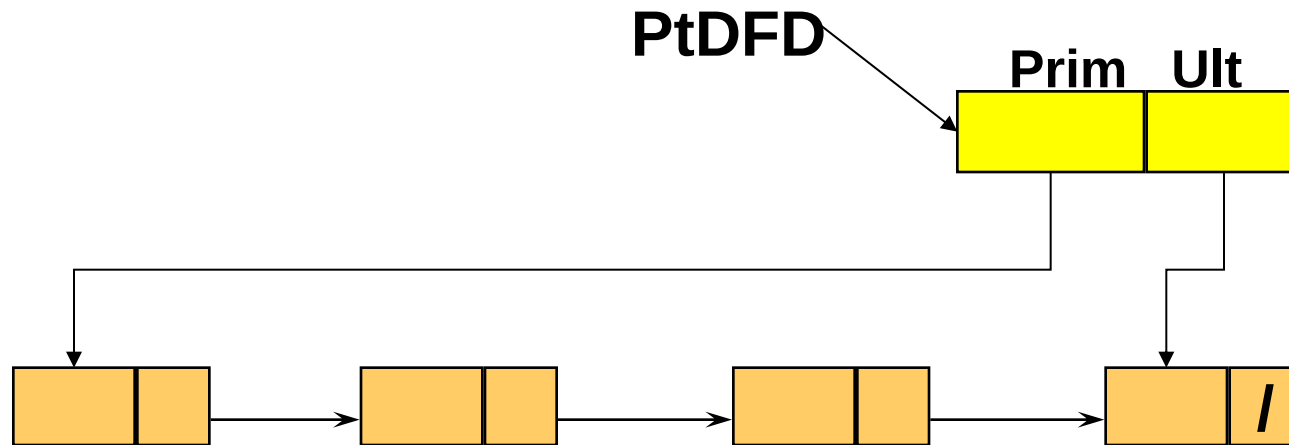
Deque (Fila Dupla)

Fila dupla implementada
por encadeamento

Fila dupla implementada por encadeamento

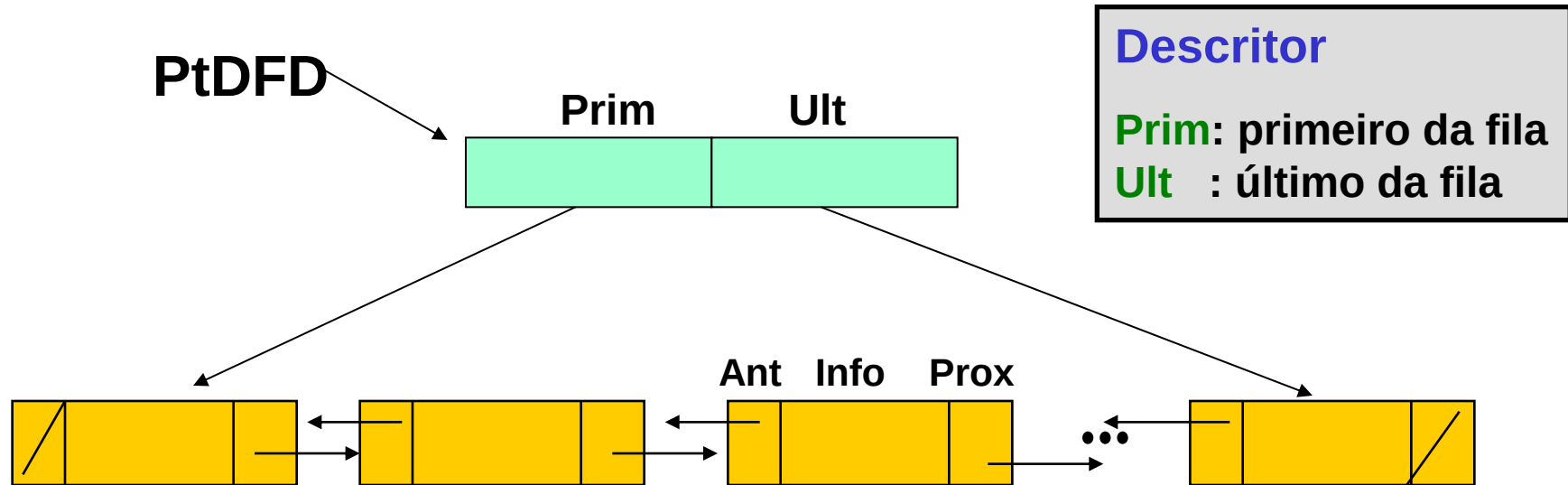


Fila dupla encadeada com descritor



Na remoção do último nodo, precisa atualizar descritor que passará a apontar para o penúltimo – percorrer a fila do início para chegar ao penúltimo

Fila dupla - duplo encadeamento e descritor



Tipo de dados para o descritor da fila dupla:

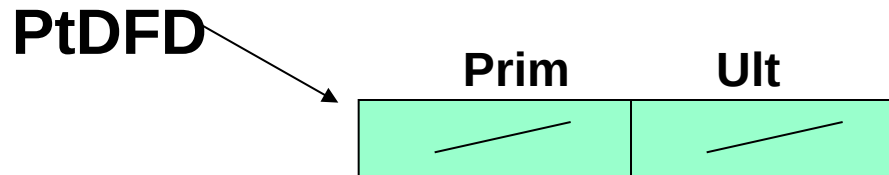
```
TipoDDeque = registro
    Prim: TipoPtNode
    Ult: TipoPtNode
fim registro
TipoPtDDeque = ↑TipoDDeque
```

Tipo de dados para nodos da fila dupla:

```
TipoNode = registro
    Ant: TipoPtNode
    Info: TipoInfoNode
    Prox: TipoPtNode
fim registro
```

Criação de fila dupla encadeada

- Alocação do descritor
- Inicialização do descritor para fila dupla vazia



Algoritmo:

Criar Deque implementada por Encadeamento

Algoritmo **CriarDequeEnc**

Entradas: -

Saída: PtDDeque (TipoPtDDeque)

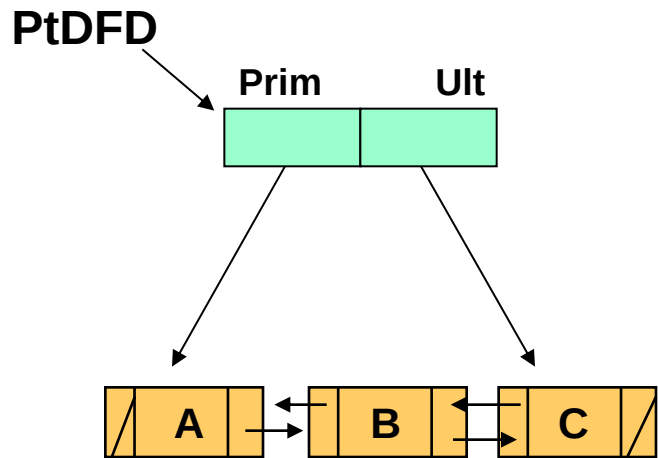
início

alocar (PtDDeque)

PtDDeque[↑].Prim $\leftarrow$ PtDDdeque[↑].Ult $\leftarrow$ nulo

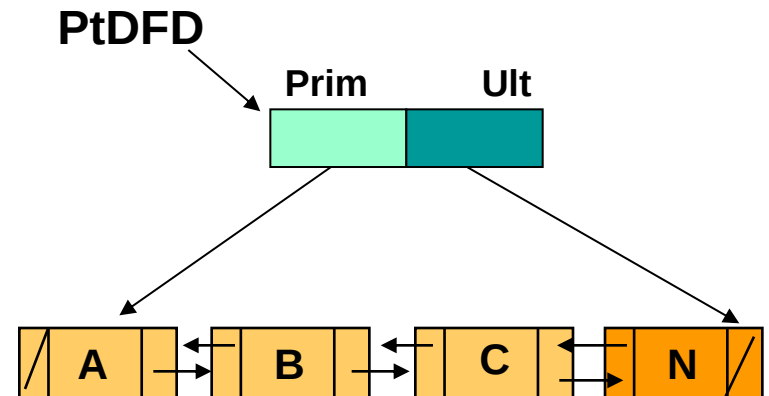
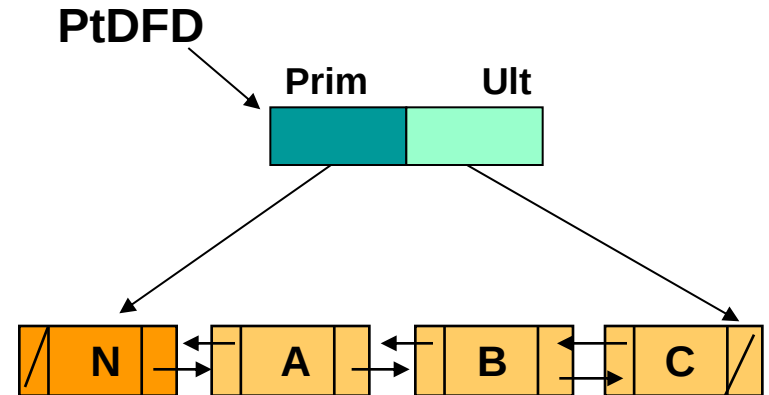
fim

Inserção de um novo nodo



No início

No final



Algoritmo:

Inserir nodo em Deque Encadeada

Algoritmo **InserirDequeEnc**

Entradas: PtDeque (TipoPtDeque)

Lado (caractere)

Valor (TipoInfoNodo)

Saída: Sucesso (lógico)

var PtNovo (TipoPtNodo)

início

Sucesso ← falso

se (Lado = "I") ou (Lado = "F")

então início

Sucesso ← verdadeiro

alocar(PtNovo)

PtNovo↑.Info ← Valor

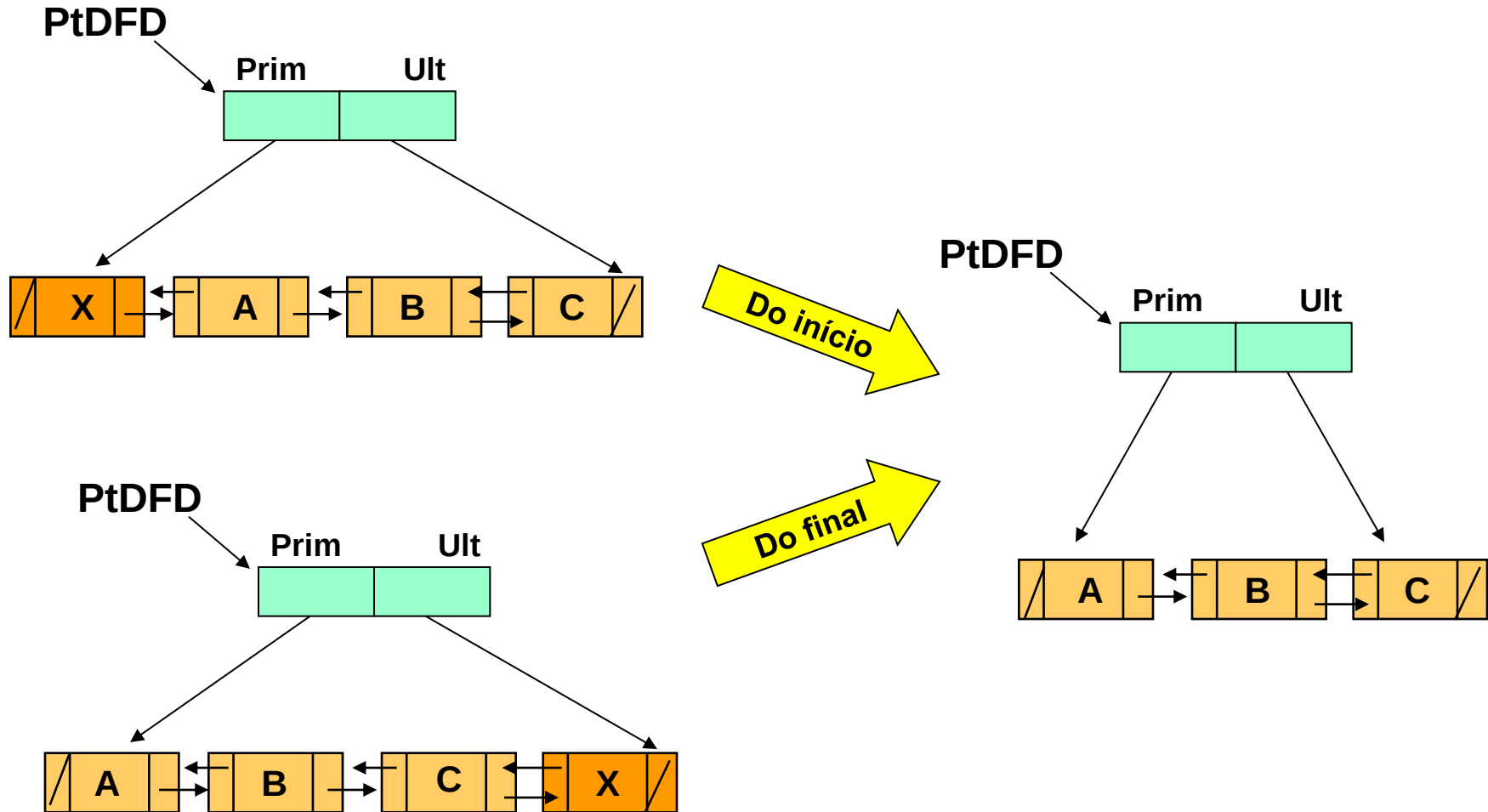
se Lado = "I"

{ então início {INSERE NO INÍCIO} SEGUE }

Algoritmo (cont): Inserir nodo em Deque Encadeada

```
então início {INSERE NO INÍCIO}
    PtNovo↑.Ant ← nulo
    se PtDDeque↑.Prim = nulo
    então início
        PtDDeque↑.Ult ← PtNovo
        PtNovo↑.Prox ← nulo
    fim
    senão início
        PtNovo↑.Prox ← PtDDeque↑.Prim
        (PtDDeque↑.Prim)↑.Ant ← PtNovo
    fim
    PtDDeque↑.Prim ← PtNovo
fim
senão início {INSERE NO FINAL}
    PtNovo↑.Prox ← nulo
    se PtDDeque↑.Prim = nulo
    então início
        PtNovo↑.Ant ← nulo
        PtDDeque↑.Prim ← PtNovo
    fim
    senão início
        (PtDDeque↑.Ult)↑.Prox ← PtNovo
        PtNovo↑.Ant ← PtDDeque↑.Ult
    fim
    PtDDeque↑.Ult ← PtNovo
fim
fim
fim
```

Remoção de um nodo



Algoritmo:

Remover nodo de Deque Encadeada

Algoritmo **RemoverDequeEnc**

Entradas: PtDDeque (TipoPtDDeque)
Lado (caractere)

Saída: Sucesso (lógico)

var PtAux, PtAnt (TipoPtNodo);

início

Sucesso ← falso

se (PtDDeque↑.Prim ≠ nulo) e ((Lado = "I") ou (Lado = "F"))

então início

Sucesso ← verdadeiro

se Lado = "I"

{ então início {**REMOVE O PRIMEIRO**} **SEGUE** }

Algoritmo (cont):

Remover nodo de Deque Encadeada

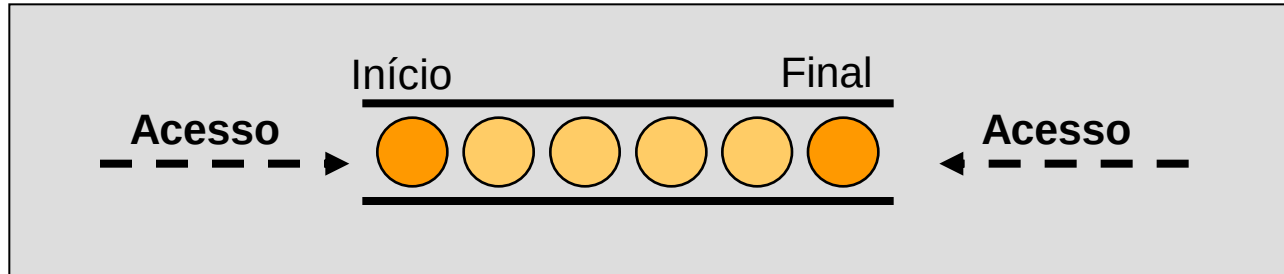
```
então início  {REMOVE O PRIMEIRO}
    PtAux ← PtDDeque↑.Prim
    se PtDDeque↑.Prim = PtDDeque↑.Ult
    então PtDDeque↑.Prim ← PtDDeque↑.Ult ← nulo
    senão início
        PtDDeque↑.Prim ← PtAux↑.Prox
        (PtDDeque↑.Prim)↑.Ant ← nulo
    fim
fim

senão início  {REMOVE O ÚLTIMO}
    PtAux ← PtDDeque↑.Ult
    se PtDDeque↑.Prim = PtDDeque↑.Ult
    então PtDDeque↑.Prim ← PtDDeque↑.Ult ← nulo
    senão início
        PtDDeque↑.Ult ← PtAux↑.Ant
        (PtDDeque↑.Ult)↑.Prox ← nulo
    fim
fim

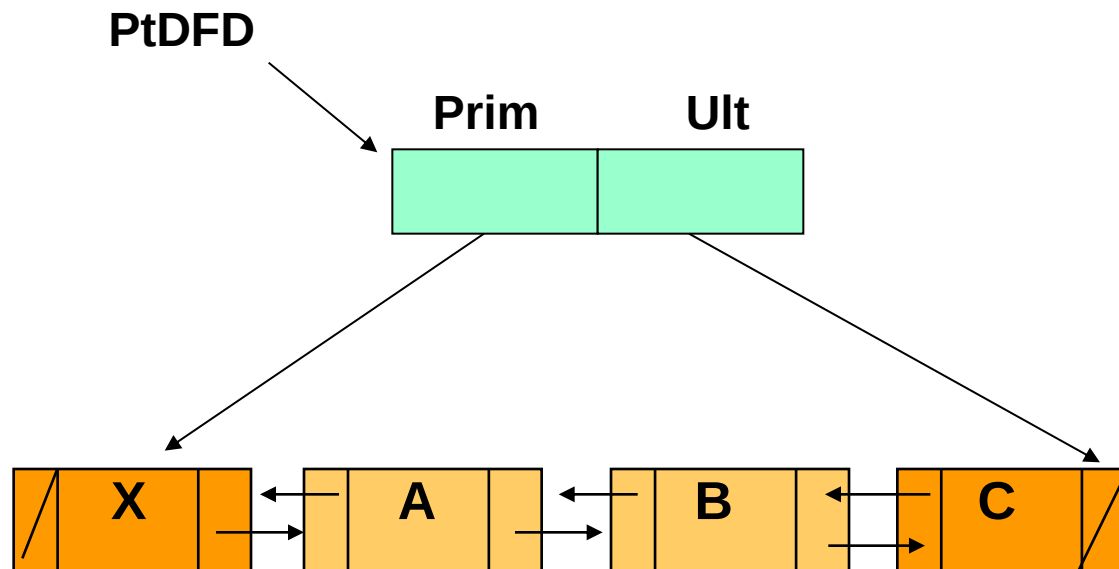
liberar(PtAux)
fim

fim
```

Acesso a fila dupla encadeada



- Apodem ser acessados o primeiro e último nodo
- Endereços diretamente no descritor



Algoritmo:

Consultar Deque implementada por Encadeamento

Algoritmo ConsultarDequeEnc

Entradas: PtDDeque (TipoPtDDeque)

Lado (caractere)

Saídas: Valor (TipoInfoNodo)

Sucesso (lógico)

início

Sucesso $\leftarrow$ falso

se (PtDDeque $\uparrow$.Prim $\neq$ nulo) e ((Lado = "I") ou (Lado = "F"))

então início

Sucesso $\leftarrow$ verdadeiro

se Lado = "I"

então Valor $\leftarrow$ (PtDDeque $\uparrow$.Prim).Info

senão Valor $\leftarrow$ (PtDDeque $\uparrow$.Ult).Info;

fim

fim